

代理工作流的批量查询处理与优化

Junyi Shen, Noppanat Wadlom, Yao Lu
National University of Singapore
Singapore
{j1shen,noppanat,luyao}@comp.nus.edu.sg

摘要

代理工作流中的大型语言模型 (LLM) 结合多步推理、异构工具使用及多个专门代理之间的协作。现有 LLM 服务引擎独立优化单次调用，多代理框架则关注编排，缺乏系统级性能规划。因此，重复提示、重叠上下文及碎片化 CPU-GPU 执行造成大量冗余和低硬件利用率，在批量分析场景中尤为突出。

我们提出将批量查询处理和优化引入代理 LLM 工作流的系统 Halo。它将每个工作流表示为结构化查询计划 DAG，并为批量查询构建合并图，显露共享计算。在联合考虑异构资源约束、预填充与解码成本、缓存复用及 GPU 放置的成本模型指导下，Halo 进行计划级优化，尽量减少冗余执行。执行处理器集成自适应批处理、KV 缓存共享及迁移、细粒度 CPU-GPU 流水线，提高整体硬件效率。

六个基准的评估表明，Halo 的批量推理最多加速 3.6 $\times$ ，在线服务吞吐量最多提升为 2.6 $\times$ ，并可扩展到数千查询和复杂图的负载，同时保持输出质量。通过统一查询优化与异构 LLM 服务，Halo 支持数据分析及决策应用中的高效代理工作流。

1 引言

大型语言模型 (LLM) 和代理工作流 [14, 23] 越来越多地用于数据分析及决策中的复杂任务。代理 LLM 工作流是目标驱动的操作序列，包含多次 LLM 调用，可能采用不同模型、提示和角色，通过协作产生最终结果。它通常需要多步推理、工具使用和流水线执行，由用户或专门的规划 LLM 编排 [31, 53, 64, 72]。近期研究还通过多代理组合与仿真建模谈判、市场动态等复杂场景，进一步增加结构复杂度和计算需求 [13, 28, 47, 66]。

考虑一个调查异常收入下降的数据分析助手 (图 1)。主规划者分派任务：搜索者检索聚合数据，分析者进行归因，关联者关联产品事件、事故工单与发布说明，编辑者整合假设、证据及反事实检验。工作流展开为大量上下文重叠的检索和摘要调用，反复生成及修改常见 SQL 查询，例如按 SKU 统计周收入，以及获取、解析新闻等 API 调用，再聚合结果。大规模运行时，数十到数百项调查并发进行，包含频繁 LLM 调用、提示复用及中间摘要。批量分析查询在实践中很常见 [20, 76, 80]：相同工作流可复制到不同市场、产品线或时段，复用查询模板和代理结构，但使用不同数据洞察作为上下文。

LLM 服务引擎通过推测解码和优化注意力核函数显著降低单请求延迟 [27, 30, 32, 43, 78]，但大多独立优化各工作流 DAG，忽略 SQL、HTTP、解析等异构 CPU 工具算子。LangGraph、AutoGen、AgentScope 等多代理框架简化工作流构建，却通常将性能交给下游服务和编排，而非做计划级优化 [13, 28, 66]。Dask、Spark 等通用 DAG 数据系统能执行 DAG 计划，但将算子视作黑盒，无法利用自适应预填充/解码调度、KV 缓存复用等 LLM 特有机会 [40, 52, 74]。Gavel [42]、Hadar [58] 等异构感知调度器主要面向多 GPU 集群中原子化深度学习训练作业，优化异构加速器分配，未联合优化代理 DAG 批次内部混合的 CPU-GPU 算子。因此，现成方案执行批量代理工作流仍低效，需要能够在 LLM 与工具混合算子下，跨 DAG 联合优化 CPU-GPU 的运行。

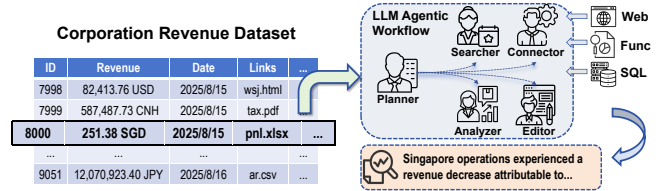


图 1: 代理工作流示例：多个 LLM 代理协作分析收入数据，为企业提供决策支持。

本文提出 Halo¹，将批量查询处理和优化扩展到数据分析及决策中的异构代理工作流。现有引擎和框架通常独立优化单查询或单工作流；Halo 则将每个工作流建模为结构化查询计划 DAG，其节点分别是 GPU 上的 LLM 调用或 CPU 上的外部函数调用。这使 Halo 能借鉴数十年的数据库优化原则，同时显式考虑 LLM 推理与工具算子的不同执行特征。

Halo 的处理器和优化器基于两点洞见。首先，Halo 直接将优化和执行表述为异构工作节点上的依赖感知放置与调度问题，而非依赖规则驱动图重写或 Volcano 风格优化流水线 [16]。每个工作流编译为具有显式先后约束的 DAG；算子包括 GPU 上的 LLM 预填充/解码，以及运行时间、竞争特性与推理明显不同的 CPU 工具，例如数据库查询、检索、解析和 API 调用。Halo 用刻画以下因素的模型决定放置与执行顺序：(i) 预填充与解码相互作用；(ii) 公共前缀的 KV 缓存复用和共享；(iii) CPU 工具竞争及通信开销。直接优化简化架构，并在实践中取得接近最优的性能。其次，Halo 将自适应批处理、缓存共享和预取、计算通信重叠等先进服务技术，整合进尊重算子依赖的统一 DAG 调度器，避免 GPU 因 CPU 工具积压闲置、工具结果延迟解码汇合等异常停顿。

回到收入调查示例，Halo 自动识别重复提示，跨并发代理批处理重叠的摘要请求，复用公共前缀 KV 缓存，并在多个 GPU 工作节点间流水执行预填充和解码；同时，在依赖及资源约束下并发调度 CPU 工具。这种整体计划级优化改善端到端延迟和吞吐量。

实现的 Halo 可集成既有 LLM 后端，例如 vLLM，支持离线批量及在线流式查询。六个基准上，相比先进代理框架，批量推理最多加速 3.6 $\times$ ，相比常用引擎则超过 400 $\times$ ；在线服务吞吐量最多提升为 2.6 $\times$ 。还在最多 2K 查询及复杂拓扑的扩展负载、最多 32B 的异构模型规模 and 不同硬件代际下，验证了稳健性与灵活性。

本文贡献如下：

- 新的系统：结合 LLM 服务与查询优化，面向 GPU LLM 和 CPU 工具组成的异构工作流；据我们所知，这是首个此类系统。
- 依赖与资源感知优化器：在异构工作节点上进行受先后关系约束的放置与调度，并提供用于最优求解的动态规划算法。

¹源码位于 https://anonymous.4open.science/r/Halo_Demo-BC86/。

研究项目	代理数量
Generative Agents [46]	25
Project Sid [2]	500
AI Metropolis [69]	1,000
生成式代理模拟 (Generative Agent Simulations) [47]	1,052
LMAgent [37]	10,000

表 1: 多代理研究项目规模不断增长, 产生大量并发, 因而需要高效批处理。

- 查询处理器: 在尊重 DAG 依赖的前提下, 支持自适应跨工作流批处理、KV 缓存共享及预取、计算通信重叠。
- 全面评估: 在不降低输出质量的前提下, 显著改善离线批量和在线流式查询的延迟与吞吐量。

2 背景与动机

本文聚焦基于工作流的代理, 这一范式广泛用于企业应用, 以落实结构化推理或标准操作流程 [7, 25]。将应用建模为 DAG $G = (V, E)$, 每个节点 $v \in V$ 是可调度计算单元, 每条边 $(u, v) \in E$ 表示数据或控制依赖 [6, 56]。这里用户请求展开为拓扑静态的结构化 DAG 执行, 不是开放式自主循环。

具有异构资源需求的节点。代理节点主要分两类: (i) LLM 节点, 对提示调用 LLM, 生成文本输出, 通常在 GPU 上运行; (ii) 工具使用节点, 通过函数调用执行通用计算或检索任务, 通常在 CPU 上运行 [53, 72], 包括结构化 SQL 查询、HTTP 等外部 API 及本地函数。每个节点 v 具有类型 $t(v) \in \{\text{LLM}, \text{Tool}\}$ 及配置, 例如 LLM 解码参数或工具函数签名。

异构作业调度的机会与挑战。多代理系统快速扩展, 需要吞吐量解决方案。如表 1 所示, 近期仿真已有数千并发代理, 例如 LMAgent [37] 最多 10,000 个。这一并发水平不仅增加压力, 也释放单请求场景缺少的优化空间。真实代理负载通常非线性, 形成带分支、汇合及迭代循环的结构化 DAG [34, 51, 66], 见图 2。这带来全局调度器可利用的强局部性:

- 模型复用: 并行分支常调用同一模型, 例如多个优化步骤使用 Qwen-32B; 共同放置并批处理这些调用, 可减少反复模型加载, 改善 GPU 驻留。
- 上下文复用: 子节点通常扩展父节点上下文, 在同一工作节点保留谱系, 可进行前缀缓存和 KV 复用 [78], 避免对共享历史重复预填充。
- 工具合并: 不同代理及会话常发出相同或重叠工具调用, 例如 SQL 检索查询; 合并为一次物理执行可摊薄开销, 共享复用结果 [11, 15]。

不同异构算子的优化机会不同。GPU 上的 LLM 推理主要受内存带宽约束, 多模型工作流还承担显著权重加载开销。连续批处理在一次前向传播中执行多个并发请求, 摊薄权重访问 [73]; 高并发还允许批内公共前缀仅计算一次, 再跨请求复用 KV 缓存 [78]。数据库及工具算子则常受 CPU 或 I/O 限制, 主要手段是请求合并、读取去重 [11] 和局部性感知调度。多个代理查询同一“世界状态”时, 相同并发请求很常见, 合并可避免冗余执行, 类似共享查询负载中的多路复用 [15]。将相似请求路由到同一节点, 还可复用预编译语句和缓冲池等热状态, 减少解析与磁盘 I/O [19]。

挑战。给定工作流 DAG 和异构工作节点, 代理调度明显难于传统请求级 LLM 服务或单查询数据库优化, 主要有三项挑战:

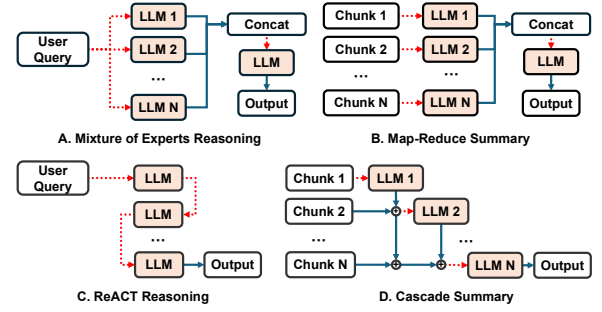


图 2: 复杂代理工作流包含模块化、协作及自适应过程。

表 2: 现有方案比较。C1: 结构感知 (全局 DAG); C2: 异构性 (CPU、GPU 运行时); C3: 有状态优化。

方案	示例	C1	C2	C3
请求级服务	vLLM, SGLang	×	×	~ [†]
数据平台	Ray, Spark	~ [‡]	×	×
应用编排器	LangGraph, AgentScope	☒	×	~ [§]
Halo (本文方案)	-	☒	☒	☒

[†] 优化内部 KV 缓存, 但缺乏跨请求逻辑感知。

[‡] 支持 DAG 执行, 但将任务视为不透明黑盒。

[§] 状态在运行时外部, 阻碍主动干预。

- C1: 结构感知。多个代理常遍历相同子图, 例如共享标准操作流程, 存在显著结构冗余。现有调度器将请求独立执行, 造成冗余 I/O 和计算。感知工作流的调度器需识别全局模式以合并请求, 不能只优化单节点。
- C2: 异构流水线空隙。执行交替经过 CPU 工具或数据库调用, 以及 GPU LLM 推理。既有 DAG 机器学习调度器 [40, 61] 常忽略这一特性, 使 GPU 等待上下文而闲置。降低延迟需要精确编排异构流水线。
- C3: 有状态的 LLM 算子。执行成本随工作节点状态显著波动。模型切换产生昂贵权重加载成本, 即模型状态; 缺少热 KV 缓存或工具连接则增加推理延迟, 即上下文状态。

表 2 比较现成方案应对这些挑战的能力: (1) LLM 服务引擎, 如 vLLM、SGLang、TGI [21, 27, 78], 在 GPU 上高效连续批处理, 但视请求为独立单元, 无法逻辑去重; (2) 带工作节点调度器的通用数据系统, 如 Ray [40], 粗粒度分配资源, 基本不感知工作流, 未利用 DAG 依赖共同放置或复用状态; (3) 应用级编排器, 如 LangGraph、AgentScope [12, 28], 捕捉代理交互逻辑, 却与服务运行时解耦。缺乏跨会话全局视图, 使它们无法跨会话批处理, 将可合并工作分割为多个 API 调用, 限制并发扩展。此外, 它们都不善于联合处理 CPU 工具和 GPU 推理, 常使 GPU 在 CPU 执行期间闲置。

3 系统概览

本文通过整体优化应对上述挑战, 联合将批量代理工作流 DAG 调度到异构 CPU-GPU 工作节点, 而非独立优化各请求。图 3 给出 Halo 的经典解析器、优化器、执行处理器架构:

解析器。读取 YAML 声明式工作流, 转为带类型的中间表示 [29] (GraphSpec)。关键变换是依赖解耦: 将嵌入的数据库查询、外部 API 和本地辅助函数等非 LLM 计算提取成独立节点。这样,

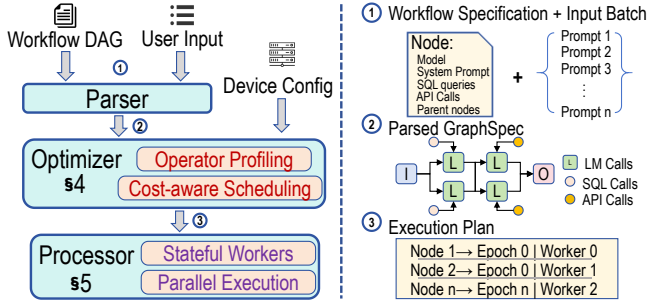


图 3: Halo 系统概览。

CPU 操作成为可调度单元，而非 LLM 提示内不可见的副作用，支持第 2 节的联合优化。

优化器。接收 GraphSpec，通过两阶段产生 ExecutionPlan。首先，算子性能分析器评估工作流所有 LLM 及非 LLM 算子 [38, 39]，估计资源消耗和延迟，建立全面的数据驱动成本基础 [54]。其次，求解器据此执行全局优化，详见第 4 节。传统优化器假定成本固定，而 Halo 联合决定：(i) LLM 节点放置，以最大化上下文复用；(ii) 非 LLM 算子执行策略；(iii) 精确任务顺序，以均衡 CPU-GPU 负载并形成高效批次。

执行处理器。作为运行时引擎，协调长期运行的工作节点池：GPU 节点承载 vLLM 实例进行 LLM 推理，CPU 线程执行数据库查询、外部 API 和本地函数等工具任务。根据给定 ExecutionPlan，编排异构节点上的任务分发、批处理及结果收集。为降低运行时开销，采用动态流水线并行 [41] 创建不同执行流，只要数据依赖允许，GPU 与 CPU 节点便并发运行 [3, 5]。

适用范围。本文方案有以下范围限定：

- 保持语义。要求与原执行严格等价，不使用代理模型等近似技术，也不修改提示。
- 单机、多 GPU。聚焦单服务器，以便细粒度、透明地控制 GPU 内存、批处理及数据局部性。
- 模型与工具使用。面向多类 LLM 模型与 CPU 密集工具交替调用的工作流。调度器需主动管理模型权重以减轻切换惩罚，同时编排 CPU 资源，避免工具执行成为推理流水线瓶颈。

4 优化器设计

将 Halo 的优化器表述为代理 LLM 工作流上以 GPU 为中心、感知资源的放置与调度问题。运行时执行含 GPU LLM 与 SQL、HTTP、解析等 CPU 工具的异构图，但优化器的显式决策变量聚焦稀缺 GPU 工作节点。CPU 工具通过隐式编排策略处理，以性能分析得到的准备成本计入目标函数。

基于调度轮次的规划。直接优化连续开始时间，搜索空间难以用于在线规划。参照离散调度研究 [26, 63]，将规划划分为决策窗口，即调度轮次 (epoch) [17, 24, 75]。轮次 e 开始时，根据当前状态选出要在 GPU 节点启动的一组 LLM 算子；执行处理器（第 5 节）随后通过细粒度流水线及机会性运行实现计划。

令 $G_{\text{LLM}} = (V, E)$ 为 LLM 算子的依赖 DAG。轮次 e 开始时，系统状态为：

$$S_e = (D_e, H_e).$$

其中， $D_e \subseteq V$ 是已完成 LLM 节点集合，而：

$$H_e = \{h_w^e \mid w \in W\}$$

汇总持久的 GPU 工作节点上下文。 h_w^e 包括：(i) 节点 w 驻留的模型权重；(ii) 其热 KV 缓存状态的紧凑表示，两者都显著影响后续 LLM 延迟。执行轮次动作后，完成集合更新为 $D_{e+1} = D_e \cup B_e$ ，工作节点上下文确定性地转移到 H_{e+1} ，其中 B_e 为本轮选中的节点。

目标。对于最终完成全部执行的有限轮次动作序列，即状态转移序列 $\mathcal{A} = \{A_0, A_1, \dots, A_{E-1}\}$ ，Halo 最小化总轮次成本：

$$\min_{\mathcal{A}} \sum_{e=0}^{E-1} C_{\text{epoch}}(S_e, A_e).$$

其中， E 是全部 LLM 节点完成所需轮次数。轮次动作 A_e 给出各工作节点的估计时间预算。令 $\phi_e(v)$ 为节点 v 分配到的工作节点，并令：

$$T_w = \sum_{v \in B_e : \phi_e(v)=w} T(w, v, S_e),$$

其中， $T(w, v, S_e)$ 是状态 S_e 下工作节点 w 执行 v 的性能分析与状态感知延迟估计，见第 §4.1 节。轮次成本定义为：

$$C_{\text{epoch}}(S_e, A_e) = \mu \cdot \max_{w \in W} T_w + (1 - \mu) \cdot \sum_{w \in W} T_w + \lambda \cdot g(A_e).$$

$\max$ 项通过降低瓶颈节点耗时改善完工时间； $\sum$ 项惩罚系统总负载，避免优化器忽略关键路径外的低效； $g(A_e)$ 则正则化分发、协调等每轮固定开销，避免计划过度碎片化。

动作空间。在轮次 e 中，优化器选取调度策略 $A_e = (B_e, \phi_e)$ ，其中：

- $B_e \subseteq V \setminus D_e$ ：轮次 e 启动的 LLM 节点集合；
- $\phi_e : B_e \rightarrow W$ ：所选节点到 GPU 工作节点的映射。

设计显式表示 GPU 分配，因为它是多模型、多代理服务的主要约束，同时允许运行时以流水线并行重叠 CPU 工作。决策 A_e 可行需满足：(1) 先后约束： B_e 构成有效拓扑截面；对每个 $v \in B_e$ ， v 的每个直接 LLM 前驱都属于 D_e 或 B_e ，使依赖步骤可在同一轮内由运行时按依赖顺序串行执行。(2) 资源约束：每个 GPU 工作节点同时最多承载一个并发 LLM 执行；为使单机场景搜索可处理，约束 ϕ_e 为单射，即 $|B_e| \leq |W|$ ，同一工作节点不分配两个作为本轮主要启动任务的节点。轮内处理器仍可采用细粒度流水线及批处理，但优化器抽象保证启动集合在物理上可实现且尊重依赖。

给定 GPU 调度 A_e 后，处理器自动推导并分发为所选 LLM 节点提供输入的前置 CPU 工具。具体按 DAG 深度优先处理 CPU 任务，较浅依赖优先，类似基于秩的列表调度，倾向于提前解决关键路径前置任务并改善 CPU-GPU 重叠。其成本通过第 §4.1 节的准备项反映在 $T(w, v, S_e)$ 中，使优化器预留足够提前量，避免 GPU 缺少工作。

动态规划求解器。连续时间的精确表述，例如 MILP，虽然追求全局最优，但面临 NP 完全性障碍 [63]，不能在线使用。HEFT [61] 或 SRPT [4] 等贪心启发式虽快，却只看局部，忽略 CPU 上下文准备等隐性成本。

Halo 通过离散化近似弥合两者差距，同时保证最优性。将状态空间裁剪到拓扑前沿后，实践中的复杂度由指数级降低到近线性，比 MILP 快数个数量级，使全局规划可处理，同时保留高效流水线编排所需的透明状态信息。

具体采用基于轮次的动态规划，在 H_e 中显式捕捉驻留权重及 KV 缓存等有状态 GPU 效应，满足马尔可夫性：过去只通过

算法 1: Halo: 异构 LLM 工作流的整体调度

输入: DAG $G = (V, E)$; 工作节点 $\mathcal{W}$; 初始状态 H^0
输出: 最优策略 Π^*
全局: 表 $M \leftarrow \emptyset$; $c^* \leftarrow \infty$
函数 $SOLVE(D, H, e)$
 若 $D = V$ **则** **返回** $(0, \emptyset)$ // 终止
 若 $(D, H) \in M$ **则** **返回** $M[(D, H)]$ // 命中记忆表
 $\mathcal{F} \leftarrow \text{GetFrontier}(G, D)$ // 拓扑就绪集合
 对每个 批次 $B \subseteq \mathcal{F}$ **及映射** $\phi: B \rightarrow \mathcal{W}$ **执行** // 搜索
 $c_{\text{now}} \leftarrow \text{CostModel}(B, \phi, H, \hat{T}_{\text{prep}}, \hat{T}_{\text{model}}, \hat{T}_{\text{infer}})$
 $H_{\text{next}} \leftarrow \text{UpdateWorkerState}(H, \phi)$ // 更新状态
 $(c_{\text{fut}}, \Pi_{\text{rest}}) \leftarrow SOLVE(D \cup B, H_{\text{next}}, e+1)$
 若 $c_{\text{now}} + c_{\text{fut}} < c^*$ **则** $c^* \leftarrow c_{\text{now}} + c_{\text{fut}}$;
 $\Pi^* \leftarrow \{(B, \phi)\} \cup \Pi_{\text{rest}}$
 $M[(D, H)] \leftarrow (c^*, \Pi^*)$
 返回 (c^*, Π^*)
主过程: **返回** $SOLVE(\emptyset, H^0, 0)$ // 触发全局规划

当前 (D_e, H_e) 影响未来成本。令 $V(S)$ 为状态 S 到全部完成的最小累计成本，贝尔曼递推为：

$$V(S) = \min_{A \in \mathcal{A}(S)} (C_{\text{epoch}}(S, A) + V(\text{NEXT}(S, A))),$$

其中， $\mathcal{A}(S)$ 枚举从 S 可达且限制于有效拓扑前沿的可行动作； $\text{NEXT}(S, A)$ 对 (D, H) 应用确定性更新。

算法 1 以记忆化递归实现此递推：枚举前沿批次及工作节点分配，用成本模型评估各候选，模拟节点状态转移，并复用缓存的子问题解，避免重复搜索。核心 $SOLVE$ 在完成集合 D 与硬件上下文 H 定义的状态空间中搜索，用哈希表 M 缓存最优子路径。 GetFrontier 将候选限制在有效拓扑截面，保证依赖约束。 CostModel 根据第 4.1 节性能指标计算 C_{epoch} ； UpdateWorkerState 模拟模型加载、缓存逐出等确定性 GPU 状态转移，确保计划物理一致。

困难性与复杂度。 令 $N_V = |V|$ 为 LLM 节点数， $M = |\mathcal{W}|$ 为 GPU 工作节点数， N_m 为模型种类数。最坏情况下，完成集合的 2^{N_V} 种可能使状态空间随 N_V 指数增长，再乘上各工作节点上下文可能性，主要由模型驻留与缓存标识决定，上界为 $O(2^{N_V} \cdot (N_m N_V)^M)$ 。实际可达集合小得多：有效状态受 DAG 拓扑限制，主要随最大前沿宽度而非总长度增长。单机的 M 通常很小，例如 ≤ 8 张 GPU；前沿宽度受工作流并行性限制，因此搜索仍可处理，并保留模型切换与 KV 优化需要的透明状态信息。

4.1 成本建模

Halo 的求解器用状态感知延迟估计 $T(w, v, S_e)$ 评估轮次动作，既反映代理工作流 CPU-GPU 异构性，也捕捉模型驻留和 KV 复用等主要有状态 GPU 效应。

状态感知延迟估计 (T)。 对轮次 e 中分配到 GPU 工作节点 w 的 LLM 算子 v ，端到端延迟建模为两项相加：(a) 前置 CPU 准备；(b) 以节点状态 h_w^e 为条件的 GPU 执行：

$$T(w, v, S_e) = T_{\text{prep}}(v) + T_{\text{gpu}}(w, v, h_w^e). \quad (1)$$

准备项 $T_{\text{prep}}(v)$ 包括上游 SQL、HTTP、本地函数等工具及使 v 可在 GPU 执行的数据物化。GPU 项依赖持久上下文 h_w^e ，汇总

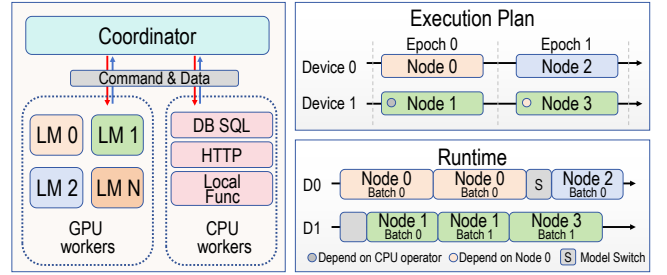


图 4: Halo 的执行处理器将执行计划映射为异构 CPU、GPU 工作节点上的协同执行。

为 $h_w^e = (m_w^e, u_w^e)$: m_w^e 为驻留模型权重， u_w^e 为热 KV 缓存状态的紧凑签名。

进一步将 T_{gpu} 分为模型切换和推理两项：

$$T_{\text{gpu}}(w, v, h_w^e) = T_{\text{model}}(v, m_w^e) + T_{\text{infer}}(v, u_w^e). \quad (2)$$

多模型工作流中，工作节点加载与当前驻留 m_w^e 不同的模型权重时，开销很大。若 m_w^e 匹配 v 所需模型，令 $T_{\text{model}}(v, m_w^e) = 0$ ；否则计入测得的重载惩罚，包括接纳新模型所需逐出及内存管理成本。这鼓励求解器连续安排同模型执行，避免异常抖动。

推理延迟取决于提示长度、生成长度及能否复用先前执行谱系的 KV 缓存。将 $T_{\text{infer}}(v, u_w^e)$ 建模为包含预填充与解码工作的状态条件估计。若 v 的提示与 u_w^e 中缓存前缀重叠，则减少有效预填充词元，施加前缀缓存折扣；否则计入完整预填充。该项奖励在同一工作节点保留谱系，引导局部性感知分配。

因此，式 (1) 显式考虑 CPU 准备。在 $T(w, v, S_e)$ 中计入 $T_{\text{prep}}(v)$ ，使优化器承担工具输入未准备好便启动 GPU 工作的风险。准备不足却过量承诺 GPU 的候选动作会立即变得昂贵，因而减少工具积压造成 GPU 空闲的流水线空隙。

算子性能分析与在线校准。 Halo 采用轻量分析实例化 T_{prep} 、 T_{model} 、 T_{infer} 三项，避免显著规划开销。

- **数据库算子。** SQL 准备使用 DBMS 的 EXPLAIN，获取索引扫描、顺序扫描等物理计划成本估计，预测当前模式和统计信息下的执行时间。
- **黑盒工具与外部 API。** HTTP 和用户函数按归一化算子与参数签名分类，维护近期调用的移动平均，可选维护有界方差估计。
- **LLM 推理。** 为每模型、硬件组合维护校准后的吞吐量曲线，将有效预填充词元和解码词元映射为延迟。按 u_w^e 指示的匹配缓存前缀长度减少有效预填充词元，反映前缀复用。

持续更新性能统计，使处理器能在竞争、网络抖动及解码方差等运行条件漂移时改进成本估计，同时保留求解器的轮次级抽象。

5 执行处理器设计

执行处理器将优化器的 EXECUTIONPLAN 实现为异构 CPU、GPU 工作节点上的高吞吐执行。设计目标是在保持计划的全局放置与排序意图的同时，通过异步编排和机会性重叠，吸收工具调用拖尾、解码时间差异、网络抖动等运行时随机性。

协调器与执行器。 执行处理器采用非阻塞、事件驱动的协调器，维护节点就绪、完成、数据血缘等全局 DAG 状态，并按照轮次计划分派工作。协调器运行一个尽量少用锁的事件循环，消

费完成事件，将新就绪节点加入按类型划分的队列。运行时管理两类常驻工作节点：(i) 有状态 GPU 执行器，各自承载一个 LLM 引擎实例，例如 vLLM 式连续批处理，并保持持久上下文 h_w ，包括驻留权重与热 KV 状态；(ii) 通用 CPU 执行器，执行 SQL、HTTP、本地函数等工具算子，并物化中间结果。

依赖解析与 CPU 调度。为避免 GPU 饥饿，协调器优先处理能够解锁当前 GPU 前沿的 CPU 任务。具体而言，按照就绪工具节点到下一个尚未满足条件的 LLM 节点的 DAG 深度递增排序，先执行层次更浅、更关键的前置任务。该策略与优化器将准备时间计入 T_{prep} 的成本模型一致，无需求解器显式调度 CPU 线程，即可改善 CPU 与 GPU 的执行重叠。

工具执行主要受 CPU 与 I/O 限制；无差别启动任务可能产生队头阻塞。执行处理器对每个工具后端限制并发数，例如每个数据库连接池或 HTTP 客户端；队列超过配置阈值时施加背压，从而稳定突发负载下的延迟。

通过请求合并减少工作量。除执行任务外，处理器还减少并发工作流之间的重复工作。对于工具算子，将算子类型及规范化参数组成的签名标准化，把签名相同的待执行任务合并为一次物理执行，再将结果分发给所有依赖该结果的逻辑节点。高扇出代理反复发出相同 SQL 模板或相同检索请求时，这种合并尤其有效。

对于结构化数据库，Halo 还在轮次内利用预备语句：将频繁重复的查询模板编译一次，在合并调用间复用预备形式，以降低高并发下的解析和计划开销。

GPU 执行：批处理、串接与重叠。每个轮次内，GPU 执行器接收计划分配的 LLM 节点，再向服务引擎提交请求；引擎在内部进行连续批处理，以提高 GPU 利用率。协调器保持计划的放置决策，以维持模型驻留和缓存复用，但不在工作节点之间施加刚性屏障同步。为改善端到端延迟，处理器采用波前式执行：LLM 节点完成后，只要其依赖后继的工具前提已满足，就立即将后继提升为就绪状态。这使快速请求无需等待更慢的兄弟节点即可向下游推进，提高流水线周转效率并减少扇入延迟。

运行时波动确实可能暂时阻塞计划中的高优先级任务，例如外部 API 出现拖尾。为避免空转，协调器可以机会性地执行缓冲区中的其他就绪任务，条件是：(i) 数据依赖满足；(ii) 执行不会破坏即将使用的 GPU 状态，例如迫使近期计划节点所需的模型被驱逐。实际中，该策略相当于一种受约束的工作窃取机制，在保持计划局部性意图的同时掩盖尾部延迟。

正确性与计划遵循。执行处理器保持语义：绝不在前驱完成前执行节点；仅对签名完全相同的算子进行合并，以确保一次物理执行等价于多次逻辑执行。启用机会性执行时，协调器只允许不违反依赖、不损害计划资源保证的动作，从而保持优化器的全局调度。

6 评估

下面全面评估 Halo 的系统性能，并与先进的 LLM 推理引擎比较。评估旨在验证以下目标：

- G1. Halo 能高效执行批量代理 LLM 工作流。对多样化负载，展示相较先进 LLM 与代理服务系统的实际墙钟延迟改善（第 6.2 节）。
- G2. Halo 的优化器以较低开销寻找近似最优计划。验证基于轮次的 DP 调度器在比离线 MILP 参照求解器低数个数量级的规划时间内，生成与其匹配的执行计划，并量化主动批处理和负载引导机制的关键贡献（第 6.3 节）。

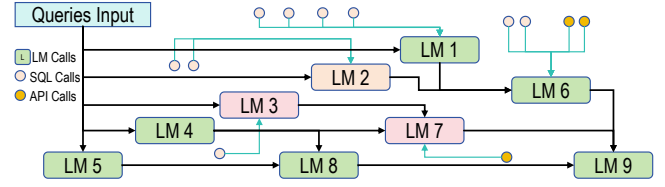


图 5: 工作流示例 W6。

- G3. Halo 为在线服务提供更高吞吐量。展示系统在连续流式条件下保持较高每秒查询数 (QPS)，利用流水线并行，在复杂工具使用工作流和线性 LLM 链上超过基线（第 6.4 节）。
- G4. Halo 具有稳健的可扩展性与硬件适应性。验证系统在不同并发水平下保持效率，即规模不变性；随着工作资源增加近似线性扩展，即弹性；并适应不同模型规模和设备拓扑（第 6.5、6.6 节）。

6.1 实验设置

测试平台与模型。所有实验在专用服务器上运行，配备两颗 AMD EPYC 9755 CPU、2.2 TiB 主存，以及三张 NVIDIA H200 NVL GPU，每张具有 141 GiB HBM。采用三个大语言模型评估系统：Qwen3 的 14B、32B 版本，以及 GPT-OSS 20B [1, 70]。

基线。选择具有代表性的 LLM 服务引擎和代理工作流系统，覆盖从以推理为中心的服务到工作流编排的不同设计方案，与 Halo 比较：

- vLLM（仅推理）：先进推理引擎，通过连续批处理优化单个请求 [27]。我们以逐查询方式实现代理工作流：客户端驱动先完整执行一个查询的工作流 DAG，再开始下一个查询。该基线采用串行执行方式，体现引擎本身的延迟，不享有跨请求批处理的收益。
- OpWise（感知拓扑的批处理）：参照 MapReduce [10] 与 Spark [74] 分阶段执行的基线。采用与 Halo 相同的批量处理器，但显式地在某一阶段执行前缓冲该阶段全部请求，以最大化批大小。然而，这种严格逐层同步禁止不同算子交错执行，拖尾任务会造成资源浪费。
- LangGraph (v1.0.4)：常用的图编排框架 [28]。将工作流表示为显式 DAG，但把 LLM 调用作为黑盒 API 操作。根据控制流是否可执行来调度任务，缺乏与 GPU 调度器协调高效放置所需的系统级反馈。
- AgentScope (v1.0.9)：基于 actor 的多代理平台。将代理封装为通过消息传递通信的独立 actor [12, 13]，优化多代理并行执行以提高吞吐量。然而，严格 actor 隔离阻止全局状态共享，使执行碎片化，难以进行整体优化。
- Parrot (v0.1.0)：通过“语义变量”暴露应用语义、感知前缀的服务系统 [34]。遵循其最佳实践，一开始提交整个工作流批次，使其能够进行全局数据流分析。该系统代表先进的语义感知服务方案。
- Halo：本文提出的工作流感知服务系统，采用基于轮次的 DP 调度器，联合优化节点执行顺序与工作节点分配。

所有系统使用相同的 vLLM 推理后端 [27] 和 PostgreSQL [57] 数据库后端，以保证推理及数据库访问层面的公平比较。采样参数、解码设置等所有控制变量在系统间保持一致。Halo 原型集成 vLLM 0.11.0、SGLang 0.5.6 与 PostgreSQL 18.0。DP 调度器通过 PyO3 加载 Rust 核心，以加速内部搜索循环；使用 maturin 为目标平台构建一次。

编号: 名称	节点数	描述
W1: IMDb-Diamond	8 / 9	菱形拓扑, 并行执行连接密集的 SQL 查询, 以考察同步压力。
W2: IMDb-TripleChain	10 / 3	三条独立并行链分别处理电影、人物、制作人员数据, 最后合并为回答。
W3: FineWiki-LongChain	9 / 6	深层顺序链, 交错频繁的阻塞式数据库检索, 测试关键路径优化。
W4: FineWiki-Bridge	9 / 3	主推理链周期性加入辅助数据库查询, 存在不规则依赖插入。
W5: TPC-H-Trident	7 / 9	同时启动三个计算密集的分析 SQL 分支 (Q1-Q22) 的并行结构。
W6: TPC-H-Fanout	9 / 12	在两个阶段向多个代理广播查询参数, 聚合复杂分析指标。

表 3: 评估工作负载。节点数 (GPU/CPU) 分别表示 LLM 推理节点与 CPU 工具节点 (例如 SQL 执行) 的数量。

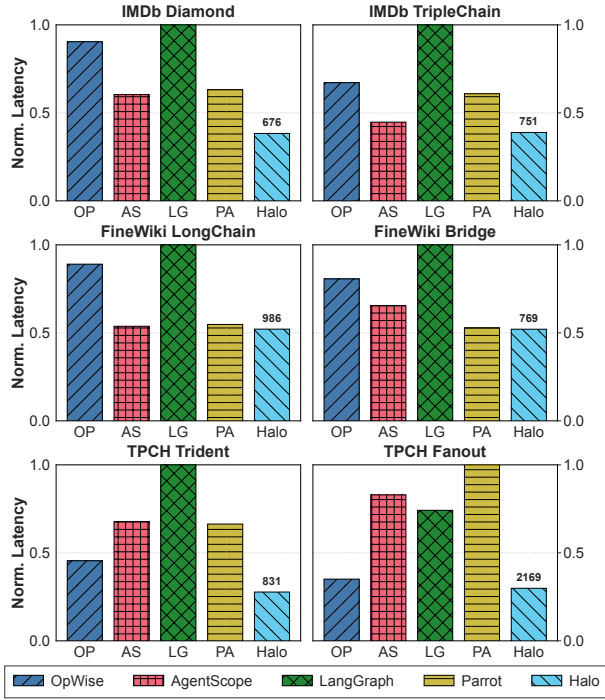


图 6: Halo 与先进基线的端到端比较。

工作负载与数据集。使用一组成但具有代表性的多阶段 LLM 工作流评估 Halo, 涵盖生产代理系统常见的结构模式。各负载表示为 DAG, 由 vLLM 服务的推理节点与异构工具节点组合而成; 工具节点执行数据库 SQL 查询或外部 API 的 HTTP 请求等 I/O 操作。总体覆盖多种依赖结构、计算与 I/O 交错方式、同步模式。

工作负载。表 3 汇总评估使用的工作负载。每个负载用于单独考察真实代理流水线中常见的一项调度挑战。图 5 进一步展示示例工作流。

部署模式构成一个关键操作差异。LangGraph、AgentScope、Parrot 等基线采用分离的客户端与服务器设计, 依赖提前启动的 vLLM 后端。这种静态要求迫使用户在执行前为所有模型类型预留 GPU 资源; 硬件受限时, 扩展到较多并发模型会受到部署操作限制。相比之下, Halo 动态配置执行上下文, 支持动态加载、驱逐模型。为确保基线具有可行的比较环境, 每个负载最多使用三种不同模型。这一约束反映基线静态部署模式的刚性, 而非 Halo 的限制; Halo 本身可处理模型异构性更高的工作流。

数据集。所有工作负载使用三个关系数据库作为数据后端, 装载公开且广泛使用的数据集, 模拟不同数据密集型代理任务。

- *FineWiki* [50]. 使用 FineWiki (HuggingFaceFW/finewiki), 它是 Wikipedia [65] 语料的高质量过滤提取版本, 保留结构格式。将精心选取的 20,000 条英语页面级记录导入 PostgreSQL, 包括元数据与原始 wikitext。为模拟点查占主导的真实检索增强生成 (RAG) 场景, 在主键和标题列上创建 B 树索引。
- *IMDb* [22]. 对于大规模结构化数据, 采用 IMDb 官方非商业数据集。将原始数据规范化为关系模式, 得到超过 200M 行的数据库。该数据集包含作品、演员、制作人员之间复杂的多对多关系。广泛为外键建立索引以支持多路连接, 模拟知识发现代理所需的复杂关系处理。
- *TPC-H* [62]. 为评估分析和决策支持负载, 使用规模因子 10 (SF=10) 生成的 TPC-H 基准。按标准模式将数据导入 PostgreSQL, 含八个核心关系、数千万个元组。工作流中的 SQL 查询来自标准 TPC-H 查询模板; 根据代理上下文动态实例化参数, 使负载在保留代理工具执行变化性的同时, 忠实体现标准分析模式。

6.2 端到端基准测试

在具有代表性的代理工作流上, 评估处理大批输入查询 ($N = 1024$) 的端到端延迟。图 6 展示比较结果。Halo 在各数据集上都取得最低延迟, 表明基于轮次的调度器能有效平衡批处理效率与执行流畅性。

比较。简单的 vLLM 逐查询执行方式, 在批量代理负载中产生难以接受的延迟。对于 W1, 1024 个请求总运行时间超过 71 小时 (2.56×10^5 s); Halo 仅需 676 秒, 实现超过 400× 加速。这一巨大差距表明, 若缺乏感知工作流的批处理, 标准推理引擎难以高效扩展以处理复杂多轮代理负载。

OpWise 最大化瞬时批大小, 但刚性逐层执行导致高昂模型重新部署成本。Halo 则优先考虑 LLM 引擎复用及 CPU、GPU 并行, 将需要相同模型的任务连续执行, 以尽量减少昂贵模型加载次数, 同时让 CPU 工具执行与 GPU 推理重叠。在 W1 上得到 2.4× 加速 (676s 对 1595s), 在复杂 W6 上维持 1.6× 优势。通过消除 OpWise 固有的频繁上下文切换和同步屏障, Halo 获得更高吞吐量。

与编排框架比较时, 同时向这些框架提交全部请求, 以最大化并发。其后端可利用引擎级批处理, 在简单工作流上具有竞争力: 在 W1-W4 上比 Halo 慢 1.03×-2.6×。但这种执行仍是盲目的: 解耦框架独立处理每个请求, 错过工作流级请求合并机会, 例如去重工具调用、合并重叠 SQL 查询。因此, 冗余较高的复杂分析负载 W5、W6 性能较差。Halo 在这两个负载上取得 2.4-3.6× 加速, 原因正是能系统地识别和合并基线分别执行的冗余 I/O 操作。此外, 解耦设计阻碍突发提交时的细粒度内存管理, 限制多模型流水线扩展。

调度器	W1			W6		
	端到端 延迟	最优性	求解 时间	端到端 延迟	最优性	求解 时间
随机 (<i>Random</i>)	367.27s	0.4	–	672.08s	0.0	–
RR (<i>Ray</i>)	351.17s	0.4	–	844.46s	0.0	–
贪心 (<i>HEFT</i>)	338.28s	0.6	–	768.18s	0.2	–
Halo	323.64s	1.0	2.24s	545.55s	1.0	1.15s
MILP (参照最优解)	322.17s	–	1.44h	544.06s	–	172.84s

表 4: 调度效率比较。“最优性”为归一化最优性评分, 定义见正文; “求解时间”表示调度算法所需的计算时间。

6.3 最优性分析

比较四种调度策略:

- Random* 将就绪算子均匀随机分派。它是忽略工作流拓扑与资源局部性的简单基线。
- RayServe-style* (RR) 遵循 Ray [40] 等现代框架采用的去中心化调度原则, 用简单轮转 (RR) 方式将算子分配给可用工作节点。虽然尊重基本依赖, 但缺乏全局协调, 常常错过优化数据局部性和下游资源争用的机会。
- HEFT* 是经典的贪心列表式 DAG 调度算法 [61]。使用局部启发式确定算子优先级, 贪心映射到使估计成本最低的资源。类似地, *JellyBean* [67] 等近期面向机器学习的系统, 也依赖贪心启发式在异构基础设施上选择和放置模型。这些方法虽考虑局部性, 贪心性质仍将视野限制在局部最优。
- Halo 使用基于轮次的动态规划调度器, 在统一的全局目标下联合优化算子排序、批处理、放置。

参照求解器及比较。参照 Alpa [33, 77] 的编译策略, 实现一个 Oracle 规划器。具体采用连续时间混合整数线性规划 (MILP) 模型求全局最优解 (S^*)。另定义以 GPU 为中心的最优性指标 $\text{Opt}(S)$, 测量一个调度与参照求解器决策的接近程度:

$$\text{Opt}(S) = \max_{\pi} \frac{|\mathcal{P}(S) \cap \pi(\mathcal{P}(S^*))|}{|\mathcal{P}(S^*)|},$$

其中 $\mathcal{P}(\cdot)$ 表示在同一 GPU 上执行的连续节点的有序对集合。该指标量化调度器保留最优执行依赖的召回率。

虽然 S^* 给出理论性能上限, 求解 MILP 的计算开销难以承受。例如, W1 的求解器收敛需要超过 1.44 小时 (5.19×10^6 ms)。Halo 仅用 2.24 秒即可计算近似最优调度, 在匹配参照方案质量的同时, 实现 2,322× 规划加速。

分析。为评估不同条件下的调度最优性, 使用两种不同负载 W1、W6 ($N = 256$)。表 4 显示, 调度最优性与归一化端到端负载延迟有强相关关系。负载拓扑越复杂, 基线与最优解的性能差距显著扩大。

发现 *Random* 与 *RR* (*Ray*) 开销较大, 在 W6 上延迟达到 1.55×。其不感知拓扑的分配方式忽略数据移动和模型切换成本, 生成低最优性分数 (0.0–0.4) 的计划。贪心 *HEFT* 在较简单 W1 上表现尚可 (1.05×), 但在复杂 W6 上显著下降 (1.41×)。这体现模型复用与流水线并行之间关键且经常冲突的权衡。贪心启发式通常过度偏好即时局部性, 以避免切换成本; 但 W6 等复杂拓扑中, 该局部优化容易无意间串行化独立分支, 或使某些节点过载, 导致资源利用不足并错失并发机会。

相比之下, Halo 在两个负载上都达到近似最优。此外, 基于 DP 的调度与 MILP 参照方案高度一致, 产生与理论最优方案结构相同的执行计划。这种一致性验证了离散化算法的合理性。

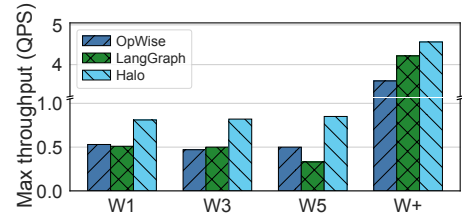


图 7: 在线吞吐量比较。Halo 在不同负载上持续超过 *OpWise* 与 *LangGraph*。

消融变体	W1	W6
	端到端延迟	端到端延迟
无性能分析评分	+20%	+8%
无 CPU 负载引导	+1%	+18%
无机会性执行	+1%	+56%
无请求合并	+16%	+154%
Halo (完整)	1.00	1.00

表 5: W1、W6 上的组件消融分析。数值越高, 移除该组件后性能下降越大。

6.4 在线服务性能

为在真实在线服务场景下评估 Halo, 向系统施加连续的异步请求流。请求进入输入队列后分派给执行引擎, 测量最大可持续吞吐量 (QPS)。其余设置与第 6.2 节相同。选择 *OpWise* 及代理框架 *LangGraph* 作基线, 展示 Halo 相对先进优化方案和常用工业工具的性能优势。除 W1、W3、W5 标准负载外, 还采用 W+: 仅含三个 LLM 节点的轻量线性链, 评估无 CPU 工具开销的快速响应场景。

图 7 显示 Halo 对两个基线均持续提供更高吞吐量。在复杂工作流 W1–W5 上, 相对 *OpWise* 加速 1.53× 至 1.74×, 例如 W3 为 0.82 对 0.47 QPS。差距来自更好的模型调度和执行并行: 相比 *OpWise* 的刚性执行, Halo 缩短空转时间。相对 *LangGraph*, 最大优势出现在 TPC-H 的 W5: 0.85 对 0.33 QPS, 即 2.58×。这是请求合并消除批处理冗余, 以及全局调度器在高并发下优化资源流水线的共同结果。即使轻量 W+ 上, Halo 的 4.57 QPS 仍超过 *OpWise* 的 3.59 和 *LangGraph* 的 4.22 QPS, 表明架构优化对简单推理链也有效。

6.5 消融研究

在 W1、W6 上进行消融, 量化 Halo 各调度与优化机制的独立贡献。表 5 报告相对完整 Halo 的延迟变化。下面讨论实验发现。

首先, 将基于历史性能分析的评分替换为简单的依赖启发式, 导致 W1 延迟增加 1.20×。简单方法把上游依赖数量等同于计算强度, 从而错误估计节点成本, 将轻量聚合节点误判为昂贵任务, 排在重计算之后。这种静态误判扰乱最优执行顺序, 造成可避免的流水线停顿。

其次, 关闭 CPU 负载引导 (T_{prep}), 导致 W6 延迟增加 1.18×。该机制对确保 CPU 与 GPU 在轮次内部负载均衡至关重要。缺少 CPU 压力信息时, 调度器倾向于让局部性优先的任务占满工作节点, 却在 GPU 空转, 在一个调度周期内产生明显资源偏斜。Halo 利用该引导调节任务派发, 保持每轮 CPU 处理与 GPU 推理平衡, 以最大化流水线吞吐量。

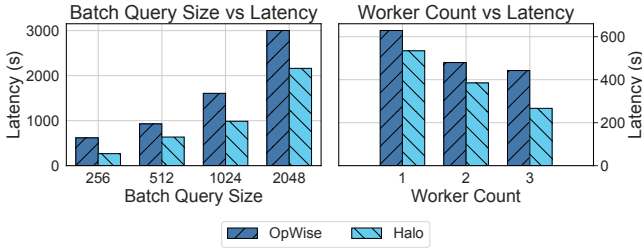


图 8: Halo 的可扩展性分析。

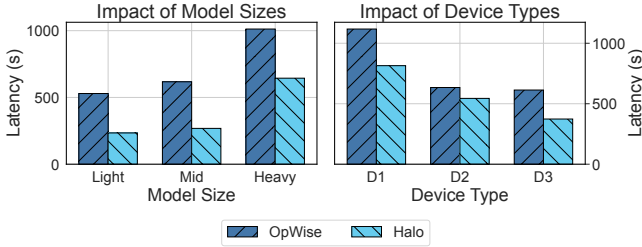


图 9: Halo 的敏感性分析。D1-D3 表示不同 GPU 配置。

移除机会性执行机制，复杂拓扑 W6 会慢 1.56×。该功能使后端执行处理器能够在 I/O 停顿等空闲周期，主动识别并执行就绪节点，前提是不违反全局依赖顺序。缺少该适应能力时，处理器受静态分派速率限制，在微小调度间隙或数据传输延迟中浪费可用 CPU、GPU 周期。

最后，不进行请求合并的变体性能退化最严重，W6 慢 2.54×。Halo 在工作流层面合并请求，扫描当前批次全部查询，识别并合并冗余 I/O，例如高扇出代理生成的相同 SQL 查询。关闭该模块后，后端必须处理原始、未削减的请求流，承受完整负载复杂度，导致严重 I/O 拥塞。

6.6 敏感性与可扩展性分析

为简化实验，使用输入规模 256 的 W3 变体评估 Halo 的稳健性和适应性。以 OpWise 为标准数据系统基线，考察不同动态环境与规模下能否保持效率。

可扩展性分析。通过改变并行度和工作节点规模评估 Halo，结果如图 8 所示。

- **查询批大小的影响。**将输入查询批大小从 256 增加到 2048。Halo 在这些规模下呈现接近线性的稳定性。OpWise 的效率确实随输入批增大而提升，但要达到 2048 才体现；该规模实际很大，可能导致内存不足 (OOM)。Halo 在不同输入批大小下保持效率优势。
- **工作节点数的影响。**使用 1 至 3 个 H200 GPU 工作节点考察 Halo 的弹性。结果表明其弹性良好，性能收益近似线性。OpWise 则因同步限制而收益递减。Halo 调度器的排队与任务分派机制确保资源弹性扩展。

敏感性分析。通过验证不同模型复杂度、异构硬件拓扑 (图 9) 和 LLM 批推理大小 (图 10) 下的适应性，评估 Halo 的稳健性。

- **模型规模的影响。**部署从轻量 (0.4B-4B) [70] 到重量 (32B 变体) [18, 60] 的模型测试适应性。结果显示 Halo 保持模型无关性。在轻量配置下，抑制调度决策开销以适应快速推理；在重量配置下，则转向最大化计算利用率。

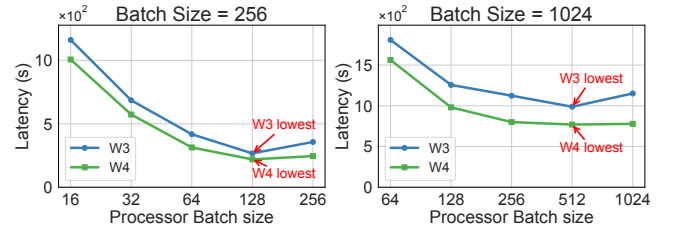


图 10: 不同执行处理器批大小下的敏感性分析。

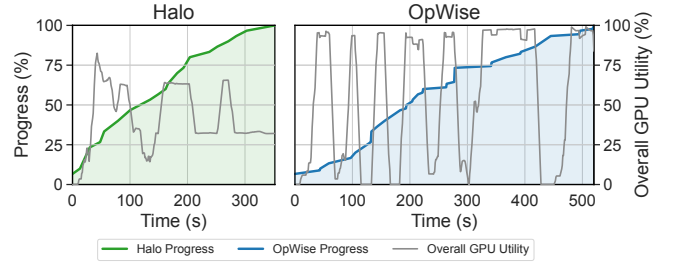


图 11: W3 (256 个输入) 的实时执行进度与 GPU 利用率。相比 OpWise 突发、低效的资源锁定模式，Halo 完成更快，资源使用更稳定。

- **设备类型的影响。**以不同 GPU 工作节点评估稳健性：D1 为 2×A100，D2 为 2×H100，D3 为 3×H200。结果显示，在各异构硬件设置中，Halo 相对 OpWise 均有收益。
- **执行处理器批大小的影响。**Halo 的执行处理器使用自己的批大小，即一次处理的 LLM 推理请求或 SQL 查询数量。在中等 256、较重 1024 查询负载下，对 W3、W4 进行敏感性分析。图 10 表明处理器批大小影响效率。开始时，硬件与数据优化协同使工作节点饱和并摊薄开销，延迟下降；但收益随后递减：批处理过大延长阶段执行，推迟跨阶段数据传播，产生使下游饥饿的队头阻塞，损害流水线流畅性。因此，Halo 寻找在不降低周转频率前提下最大化硬件利用率的拐点。

6.7 案例研究

为深入理解执行动态，对 256 个输入查询的 W3 运行行为进行性能分析。图 11 可视化并比较 Halo 与 OpWise 的实时执行进度 (彩色线) 和瞬时 GPU 利用率 (灰线)。

进度轨迹显示，Halo 的斜率更陡且更稳定，即完成速率更高。这来自更高并行度：非阻塞调度器有效重叠 GPU 推理与数据库查询等 I/O 任务，掩盖延迟，避免阻碍 OpWise 的“停下来等待”。GPU 利用率也支持这一效率结论。参照集群调度 [68] 的分析方法，观察到 OpWise 呈突发式“锯齿”模式，在接近 100% 饱和与空闲间振荡，反映低效资源锁定和数据移动。Halo 执行更稳定，空转间隙更少。

为量化成本影响，计算累计 GPU 使用量 ($\int_0^T U(t)dt$)，作为总 GPU 秒数的代理指标；GPU 秒数是云环境主要计费单位 [71]。OpWise 累计消耗 28,889 unit·s；Halo 更早完成，仅消耗 14,690 unit·s。GPU 秒数降低 $\approx 2.0\times$ ，表明高吞吐来自消除无效资源占用。

6.8 局限性

实验表明 Halo 有效，但仍在努力扩展适用范围、完善生产使用方案。当前局限如下：

- **部署范围。**Halo 面向单机多 GPU，以对白盒方式控制 GPU 内存、批处理和局部性。将相同保证扩展到多节点，需要额外的分布式放置、缓存移动、感知网络的调度机制。
- **固定计划与保持语义的优化。**Halo 假设逻辑工作流 DAG 固定，要求严格语义等价，不改写提示词、不使用近似技术。因此，无法充分采用以准确性换成本、或在线调整工作流结构的高层改写。

7 相关工作

LLM 服务与推理优化。LLM 快速部署推动推理系统创新。早期优化聚焦内存管理与连续批处理，vLLM [27] 引入 *Page-dAttention*，消除 KV 缓存碎片。为处理长上下文的内存墙，SGLang [78] 提出 *RadixAttention* 自动共享前缀；CacheGen [36] 与 LMCache [9] 等探索分离式存储以降低延迟。近期，DistServe [79] 与 Splitwise [49] 等分离架构隔离预填充与解码，解决资源干扰。然而，它们主要优化引擎或请求层面，缺少跨复杂代理 DAG 进行节点重排、算子融合等计划级优化所需的全局可见性；Halo 解决这一问题。

多代理框架。LangGraph [28]、AutoGen [66]、AgentScope [13] 等通过状态管理和消息传递抽象，简化代理应用逻辑构建。它们擅长定义代理行为，但通常将底层 LLM 推理视作黑盒 API。相比 Halo 的白盒优化，该解耦使运行时无法充分利用工作流结构依赖，产生重复计算及次优硬件利用率。

代理工作流的系统支持。近期工作开始连接框架与引擎。Ayo (前称 Teola) [59] 引入细粒度编排框架，将工作流编译为基于原语的数据流图，支持原语级流水线。类似地，Parrot [34] 通过语义变量支持 KV 复用。二者主要面向单查询延迟；Halo 则通过统一成本模型优化批处理吞吐量。此外，KVFlow [44] 通过感知工作流的驱逐改善内存效率，但仍采用反应式策略；Halo 主动将缓存状态纳入全局调度。较新的 Murakkab [8] 提出声明式、感知 SLO 的多租户代理工作流服务栈，分离工作流描述与执行配置，利用性能分析和适应性运行时选择模型、硬件并重配置执行以满足 SLO。Halo 则假定逻辑计划固定，聚焦 CPU、GPU 协同调度、算子重排、批处理等批次内物理优化，利用 DP 式规划最大化复杂依赖 DAG 执行的吞吐量。

异构调度与流水线。已有系统在集群或流水线并行层面利用硬件异构性。Gavel [42] 及后续工作 [58] 等集群调度器用吞吐模型在深度学习任务间分配异构加速器，优化总完成时间、任务完成时间及利用率。PipeDream [41]、HetPipe [45] 等执行级流水线系统，在不同性能 GPU 上优化训练阶段划分、映射，常使用 DP 式规划器减少流水线气泡。Halo 面向推理阶段混合 GPU LLM 算子和 SQL、HTTP、解析等 CPU 工具算子的代理 DAG，进行放置、重排、批处理等计划级批次内优化，平衡 CPU 与 GPU 负载，并利用缓存、工具状态。在方法上，继承流水线规划的 DP 思路，将其从线性流水线推广到复杂依赖、多模型 DAG 调度。

面向 AI 的查询优化。Halo 受数据库查询优化启发。DocETL [55]、Palimpsest [35]、LOTUS [48] 等采用声明式语言和基于成本的优化器，选择高效逻辑计划或改写流水线，以兼顾准确性、成本。这些系统主要进行模型选择、提示词改写等逻辑重构；Halo 假定逻辑计划固定，聚焦异构硬件上的调度、放置、批处理等物理执行优化。因此，可将 Halo 视为支撑这些高层逻辑优化器的高效运行时引擎。

8 结论

本文提出 Halo，一个面向代理 LLM 工作负载、感知工作流的查询处理系统。将每个请求建模为跨越 GPU LLM 算子与 CPU 工具算子的 DAG，以基于成本的调度、放置重叠执行，保持局部性，利用跨请求批处理与共享。实验表明，相比仅推理服务和已有编排框架，Halo 以较低规划开销改善端到端延迟与在线吞吐量。我们认为它为可扩展的代理分析提供了实用运行时基础。

译者注与原图文字对照

范围与数字核对。本文对象是固定逻辑 DAG 的物理执行优化。运行时通过异步执行应对时间波动，规划中使用成本估计；它未建立 Hermes 式带转移概率的未来需求图。循环如需用 DAG 表达，应先有界展开。超过 400 倍的结果比较逐查询串行 vLLM，并非所有并发服务配置；正文 W1 的约 71 小时与 676 秒之比约为 379，原文仍称“超过 400 倍”，此处保留原数并提示其不一致。其他框架在主实验中同时接收全部请求。实验为单机最多三张 GPU，不能据此推断多节点线性扩展。

最优性。动态规划优化离散轮次及估计成本构成的目标，不等于任意连续运行时随机执行的全局最优。最优性评分只衡量同一 GPU 上连续节点有序对的召回率，评分 1 不保证延迟完全相同：表中 W1 为 323.64 对 322.17 秒，W6 为 545.55 对 544.06 秒。规划耗时加速来自这些有限实例。正文将 Random 与 RR 在 W6 上都称为 1.55 倍，表中 Random 672.08 秒、RR 844.46 秒、Halo 545.55 秒，对应约 1.23 和 1.55 倍。消融 1.20 倍等指延迟变为原来的 1.20 倍，即增加 20%，并非增加 120%。

伪代码条件。原算法枚举批次时未显式排除空集，且把最优成本变量写为全局变量；作为 Bellman 递归，应要求动作使完成集合严格增长，并为各状态保存各自的局部最小值。原文的状态转移与“轮次内可含前驱”的文字定义，和伪代码只枚举就前沿的关系，也需要在实现中明确。本译文忠实保留伪代码，不据此断言实际代码存在错误。工具签名相同可安全合并，还依赖工具语义：对有副作用或时变读取，参数相同本身未必保证可合并；这是阅读其严格等价声明时需要核对的前提。

模板信息。源码含 2018 年虚构会议、占位 DOI/ISBN，以及 2007/2009 年收稿日期等 ACM 模板信息。中文版取消其显示，作者、机构、有效正文、参考文献保持。源码的实现与附录文件为空，全文无被省略的实质附录。

图 1。Planner: 规划器; Searcher: 搜索器; Connector: 连接器; Analyzer: 分析器; Editor: 编辑器; Corporation Revenue Dataset: 企业收入数据集; ID: 编号; Revenue: 收入; Date: 日期; Links: 链接; LLM Agentic Workflow: LLM 代理工作流; Web: 网页; Func: 函数; SQL: 数据库查询。示例结论“Singapore operations experienced a revenue decrease attributable to...”: 新加坡业务收入下降，原因是……; 金额、币种、文件名与日期保留。

图 2。User Query: 用户查询; LLM Output: LLM 输出; Chunk: 数据块; Concat: 拼接; A. Mixture of Experts Reasoning: 混合专家推理; B. Map-Reduce Summary: 映射归约摘要; C. ReACT Reasoning: ReACT 推理; D. Cascade Summary: 级联摘要; Output: 输出。

图 3。Workflow Specification + Input Batch: 工作流描述与输入批; Parser: 解析器; Parsed GraphSpec: 解析后的图描述; Workflow DAG: 工作流 DAG; User Input: 用户输入; Optimizer: 优化器; Operator Profiling: 算子性能分析; Cost-aware Scheduling: 感知成本的调度; Execution Plan: 执行计划; Processor: 执行处理器; Stateful Workers: 有状态工作节点; Parallel Execution: 并行执行; Device Config: 设备配置; Prompt: 提示词; Node: 节点; Epoch: 轮次; Worker: 工作节点; Model: 模型; System Prompt: 系统提示词; SQL queries/SQL Calls: SQL 查询/调用; API Calls: API 调用; LM Calls: 语言模型调用; Parent nodes: 父节点。

图 4。Coordinator: 协调器; Execution Plan: 执行计划; Runtime: 运行时; GPU/CPU workers: GPU/CPU 工作节点; Device: 设备; Command & Data: 命令与数据; Local Func: 本地函数; DB SQL: 数据库 SQL; Model Switch: 模型切换; Depend on Node 0: 依赖节点 0; Depend on CPU operator: 依赖 CPU 算子; Node: 节点; Batch: 批次; Epoch: 轮次; LM: 语言模型。

图 5。Queries Input: 输入查询; SQL Calls、LM Calls、API Calls: SQL、语言模型、API 调用。W6 中 LM1-LM9 是节点标识。

图 6。Norm. Latency: 归一化延迟; OP/AS/LG/PA 分别为 OpWise、AgentScope、LangGraph、Parrot; IMDb Diamond: IMDb 菱形; IMDb TripleChain: IMDb 三链; FineWiki LongChain: FineWiki 长链; FineWiki Bridge: FineWiki 桥形; TPCH Trident: TPCH 三叉; TPCH Fanout: TPCH 扇出。柱中数值为 Halo 实际秒数。

图 7。Max throughput (QPS): 最大吞吐量 (每秒查询数)。W1、W3、W5、W+ 是工作负载标识。

图 8。Batch Query Size: 查询批大小; Worker Count: 工作节点数; Latency (s): 延迟 (秒); Batch Query Size vs Latency: 查询批大小与延迟; Worker Count vs Latency: 节点数与延迟。

图 9。Model Size: 模型规模; Light/Mid/Heavy: 轻/中/重; Device Type: 设备类型; Impact of Model Sizes: 模型规模的影响; Impact of Device Types: 设备类型的影响; Latency (s): 延迟 (秒)。

图 10。Processor Batch size: 执行处理器批大小; Batch Size = 256/1024: 输入查询批大小为 256/1024; W3/W4 lowest: W3/W4 最低延迟点; Latency (s): 延迟 (秒); 纵轴乘以图中标明的 100。

图 11a。Time (s): 时间 (秒); Progress (%): 完成进度 (百分比); Overall GPU Utility (%): 总体 GPU 利用率; Halo/OpWise Progress: 对应系统的执行进度; Overall GPU Utility: 总体 GPU 利用率。

算法标识对照。GetFrontier: 获取就绪前沿; CostModel: 成本模型; UpdateWorkerState: 更新工作节点状态; Solve: 递归求解; ExecutionPlan: 执行计划。保留这些可与源码对应的函数及类型名。

参考文献

- [1] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925* (2025).
- [2] Altera AL, Andrew Ahn, Nic Becker, Stephanie Carroll, Nico Christie, Manuel Cortes, Arda Demirci, Melissa Du, Frankie Li, Shuying Luo, et al. 2024. Project Sid: Many-agent simulations toward AI civilization. *arXiv preprint arXiv:2411.00114* (2024).
- [3] Cédric Augonnet, Samuel Thibault, Raymond Namyst, and Pierre-André Wacrenier. 2009. StarPU: a unified platform for task scheduling on heterogeneous multicore architectures. In *European Conference on Parallel Processing*. Springer, 863–874.
- [4] Nikhil Bansal and Mor Harchol-Balter. 2001. Analysis of SRPT scheduling: Investigating unfairness. In *Proceedings of the 2001 ACM SIGMETRICS International conference on Measurement and modeling of computer systems*. 279–290.
- [5] Michael Bauer, Sean Treichler, Elliott Slaughter, and Alex Aiken. 2012. Legion: Expressing locality and independence with logical regions. In *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–11.
- [6] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nycz, et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 17682–17690.
- [7] Harrison Chase. 2022. *LangChain*. <https://github.com/langchain-ai/langchain>
- [8] Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini. 2025. Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms. *arXiv preprint arXiv:2508.18298* (2025).
- [9] Yihua Cheng, Yuhua Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. Lmcache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665* (2025).
- [10] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: simplified data processing on large clusters. *Commun. ACM* 51, 1 (2008), 107–113.
- [11] Facebook. 2019. DataLoader: Data loading utility for JavaScript. <https://github.com/graphql/dataloader>.
- [12] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, et al. [n.d.]. Agentscope: A flexible yet robust multi-agent platform, 2024. URL <https://arxiv.org/abs/2402.14034> ([n.d.]).
- [13] Dawei Gao, Zitao Li, Yuexiang Xie, Weirui Kuang, Liuyi Yao, Bingchen Qian, Zhijian Ma, Yue Cui, Haohao Luo, Shen Li, et al. 2025. AgentScope 1.0: A Developer-Centric Framework for Building Agentic Applications. *arXiv preprint arXiv:2508.16279* (2025).
- [14] Jiaxuan Gao, Wei Fu, Minyang Xie, Shusheng Xu, Chuyi He, Zhiyu Mei, Banghua Zhu, and Yi Wu. 2025. Beyond ten turns: Unlocking long-horizon agentic search with large-scale asynchronous rl. *arXiv preprint arXiv:2508.07976* (2025).
- [15] Georgios Giannakis, Gustavo Alonso, and Donald Kossmann. 2012. SharedDB: killing one thousand queries with one stone. *arXiv preprint arXiv:1203.0056* (2012).
- [16] Goetz Graefe and William J McKenna. 1993. The volcano optimizer generator: Extensibility and efficient search. In *Proceedings of IEEE 9th international conference on data engineering*. IEEE, 209–218.
- [17] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. 2014. Multi-resource packing for cluster schedulers. *ACM SIGCOMM Computer Communication Review* 44, 4 (2014), 455–466.
- [18] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [19] Joseph M Hellerstein, Michael Stonebraker, James Hamilton, et al. 2007. Architecture of a database system. *Foundations and Trends® in Databases* 1, 2 (2007), 141–259.
- [20] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiuxi Chen, and Julian McAuley. 2024. Bridging language and items for retrieval and recommendation. *arXiv preprint arXiv:2403.03952* (2024).
- [21] Hugging Face. 2023. Text Generation Inference. <https://github.com/huggingface/text-generation-inference>.
- [22] IMDb.com, Inc. 2024. IMDb Non-Commercial Datasets. <https://developer.imdb.com/non-commercial-datasets/>. Accessed: 2025-01-02.
- [23] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516* (2025).
- [24] Ram Srivatsa Kannan, Lavanya Subramanian, Ashwin Raju, Jeongseob Ahn, Jason Mars, and Lingjia Tang. 2019. Grand slam: Guaranteeing slas for jobs in microservices execution frameworks. In *Proceedings of the Fourteenth EuroSys Conference 2019*. 1–16.
- [25] Omar Khattab, Arnab Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. 2023. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714* (2023).
- [26] Yu-Kwong Kwok and Ishfaq Ahmad. 1999. Static scheduling algorithms for allocating directed task graphs to multiprocessors. *ACM Computing Surveys (CSUR)* 31, 4 (1999), 406–471.
- [27] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [28] LangChain Inc. 2024. LangGraph: Build resilient language agents as graphs. <https://langchain-ai.github.io/langgraph/>. Accessed: 2025-01-02.
- [29] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2020. MLIR: A Compiler Infrastructure for the End of Moore's Law. *arXiv:2002.11054 [cs.PL]* <https://arxiv.org/abs/2002.11054>
- [30] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.
- [31] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [32] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840* (2025).
- [33] Zhuohan Li, Lianmin Zheng, Yimin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 663–679.
- [34] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.
- [35] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpsest: Optimizing ai-powered analytics with declarative query processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)*. 2.
- [36] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [37] Yijun Liu, Wu Liu, Xiaoyan Gu, Yong Rui, Xiaodong He, and Yongdong Zhang. 2024. Lmagent: A large-scale multimodal agents society for multi-user simulation. *arXiv preprint arXiv:2412.09237* (2024).
- [38] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025).
- [39] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: A learned query optimizer. *arXiv preprint arXiv:1904.03711* (2019).
- [40] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elilbol, Zongheng Yang, William Paul, Michael I Jordan, et al. 2018. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 561–577.

- [41] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.
- [42] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. 2020. {Heterogeneity-Aware} cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 481–498.
- [43] NVIDIA. 2022. NVIDIA FasterTransformer. <https://github.com/NVIDIA/FasterTransformer>.
- [44] Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. *arXiv preprint arXiv:2507.07400* (2025).
- [45] Jay H Park, Gyeongchan Yun, M Yi Chang, Nguyen T Nguyen, Seungmin Lee, Jaesik Choi, Sam H Noh, and Young-ri Choi. 2020. {HetPipe}: Enabling large {DNN} training on (whimpy) heterogeneous {GPU} clusters through integration of pipelined model parallelism and data parallelism. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. 307–321.
- [46] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.
- [47] Joon Sung Park, Carolyn Q Zou, Aaron Shaw, Benjamin Mako Hill, Carrie Cai, Meredith Ringel Morris, Robb Willer, Percy Liang, and Michael S Bernstein. 2024. Generative agent simulations of 1,000 people. *arXiv preprint arXiv:2411.10109* (2024).
- [48] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4171–4184.
- [49] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Ñigo Gori, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [50] Guilherme Penedo. 2025. FineWiki. <https://huggingface.co/datasets/HuggingFaceFW/finewiki> Source: Wikimedia Enterprise Snapshot API (<https://api.enterprise.wikimedia.com/v2/snapshots>). Text licensed under CC BY-SA 4.0 with attribution to Wikipedia contributors..
- [51] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2024. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 15174–15186.
- [52] Matthew Rocklin et al. 2015. Dask: Parallel computation with blocked algorithms and task scheduling.. In *SciPy*. 126–132.
- [53] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
- [54] P Griffiths Selinger. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 23–34.
- [55] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G Parameswaran, and Eugene Wu. 2024. Docetl: Agentic query rewriting and evaluation for complex document processing. *arXiv preprint arXiv:2410.12189* (2024).
- [56] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yuet-ing Zhuang. 2023. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems* 36 (2023), 38154–38180.
- [57] Michael Stonebraker and Lawrence A. Rowe. 1986. The design of POSTGRES. In *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data (Washington, D.C., USA) (SIGMOD '86)*. Association for Computing Machinery, New York, NY, USA, 340–355. doi:10.1145/16894.16888
- [58] Abeda Sultana, Nabin Pakka, Fei Xu, Xu Yuan, Li Chen, and Nian-Feng Tzeng. 2025. Resource Heterogeneity-Aware and Utilization-Enhanced Scheduling for Deep Learning Clusters. *arXiv preprint arXiv:2503.10918* (2025).
- [59] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards End-to-End Optimization of LLM-based Applications with Ayo. Association for Computing Machinery, New York, NY, USA, 1302–1316. <https://doi.org/10.1145/3676641.3716278>
- [60] Qwen Team. 2025. Qwq-32b: Embracing the power of reinforcement learning.
- [61] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. 2002. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE transactions on parallel and distributed systems* 13, 3 (2002), 260–274.
- [62] Transaction Processing Performance Council. 2024. TPC Benchmark H (Decision Support) Standard Specification. <https://www.tpc.org/tpch/>. Revision 3.0.1.
- [63] Jeffrey D Ullman. 1975. NP-complete scheduling problems. *Journal of Computer and System sciences* 10, 3 (1975), 384–393.
- [64] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [65] Wikimedia Foundation. 2024. Wikipedia: Database Download. <https://dumps.wikimedia.org/>. English Wikipedia Dump. Accessed: 2025-01-02.
- [66] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.
- [67] Yongji Wu, Matthew Lentz, Danyang Zhuo, and Yao Lu. 2023. Serving and Optimizing Machine Learning Workflows on Heterogeneous Infrastructures. *Proceedings of the VLDB Endowment* 16, 3 (2023), 406–419.
- [68] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. 2020. {AntMan}: Dynamic scaling on {GPU} clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 533–548.
- [69] Zhiqiang Xie, Hao Kang, Ying Sheng, Tushar Krishna, Kayvon Fatahalian, and Christos Kozyrakis. 2025. Ai metropolis: Scaling large language model-based multi-agent simulation with out-of-order execution. *Proceedings of Machine Learning and Systems* 7 (2025).
- [70] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [71] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker, et al. 2023. {SkyPilot}: An intercloud broker for sky computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 437–455.
- [72] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
- [73] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [74] Matei Zaharia, Mosharaf Chowdhury, Michael J Franklin, Scott Shenker, and Ion Stoica. 2010. Spark: Cluster Computing with Working Sets. In *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*.
- [75] Matei Zaharia, Tathagata Das, Haoyuan Li, Timothy Hunter, Scott Shenker, and Ion Stoica. 2013. Discretized streams: Fault-tolerant streaming computation at scale. In *Proceedings of the twenty-fourth ACM symposium on operating systems principles*. 423–438.
- [76] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
- [77] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.
- [78] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
- [79] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [80] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, et al. 2024. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934* (2024).